

# Phân tích dữ liệu và dự đoán `wage_eur`

Mục tiêu: dự đoán giá trị `wage_eur` từ các biến số số học trong bảng dữ liệu FIFA 22.

Tiến trình:

1. Chuẩn hóa dữ liệu số.
2. Tính VIF cho từng biến số và chỉ giữ biến số có  $VIF < 5$ .
3. Thực hiện `bidirectional stepwise regression` với 3 tiêu chí khác nhau: `adjusted R^2`, `AIC`, `BIC`.
4. So sánh 3 mô hình và chọn mô hình tốt nhất.

## Prompt cho phần coding:

Từ nguồn data, bạn sẽ vừa viết code, vẽ biểu đồ phân tích, viết báo cáo trong cùng file ipynb.

Mục tiêu là dự đoán `wage_eur` dựa trên các biến số khác.

Để lựa chọn các biến số, thực hiện phân tích VIF cho từng biến số, rồi chỉ chọn biến số có  $VIF < 5$ . Đồng thời chỉ sử dụng các biến số dạng numerical (số tính toán được)

Sau đó, thực hiện `bidirectional stepwise regression`, với tập các biến đầu tiên là rỗng, thêm rồi bớt từng biến để so sánh trên các metric. Tạo 3 quy trình riêng sử dụng metric là  $R^2_{adj}$  (càng lớn càng tốt), hệ số AIC, BIC (càng nhỏ càng tốt).

Sau mỗi quy trình, sử dụng mô hình kết quả để tạo phương trình dự đoán `wage_eur` (giá Euro) rồi so sánh 3 mô hình cuối để chọn 1 mô hình tốt nhất cuối cùng.

Ngôn ngữ tiếng việt, kết hợp markdown và code, chỉ sử dụng numpy và pandas

## Prompt cho phần report:

Hãy bổ sung kĩ phần markdown giải thích quá trình phân tích, đồng thời vẽ biểu đồ thể hiện các chỉ số VIF, vẽ biểu đồ thể hiện tiến độ của metric được chọn cho từng quá trình `bidirectional stepwise regression`, phân tích rõ model cuối cùng và viết phương trình, phân tích lý do cụ thể.

Có thể sử dụng thêm matplotlib và seaborn để vẽ biểu đồ

```

In [41]: import numpy as np
import pandas as pd

# Đọc dữ liệu và chỉ giữ các cột số học
raw_data = pd.read_csv("FIFA 22.csv")
raw_data = raw_data.copy()

# Biên mục tiêu
target = "wage_eur"

# Chỉ lấy các cột có kiểu số học
numeric = raw_data.select_dtypes(include=[np.number]).copy()

if target not in numeric.columns:
    raise ValueError(f"Không tìm thấy biến mục tiêu {target} trong dữ liệu s

# Giữ target riêng và loại bỏ các cột khác có NaN
target_column = numeric[target]
numeric = numeric.drop(columns=[target])
numeric = numeric.dropna(axis=1, how="any")

# Đưa target trở lại và loại bỏ các hàng có NaN còn lại
numeric[target] = target_column
numeric = numeric.dropna(axis=0, how="any").reset_index(drop=True)

print("Số lượng quan sát sau khi loại cột hoặc hàng có NaN:", len(numeric))
print("Các cột số học hiện có:", numeric.columns.tolist())

numeric.head()

```

Số lượng quan sát sau khi loại cột hoặc hàng có NaN: 46

Các cột số học hiện có: ['sofifa\_id', 'overall', 'potential', 'age', 'height\_cm', 'weight\_kg', 'nationality\_id', 'nation\_team\_id', 'nation\_jersey\_number', 'weak\_foot', 'skill\_moves', 'international\_reputation', 'attacking\_crossing', 'attacking\_finishing', 'attacking\_heading\_accuracy', 'attacking\_short\_passing', 'attacking\_volleys', 'skill\_dribbling', 'skill\_curve', 'skill\_fk\_accuracy', 'skill\_long\_passing', 'skill\_ball\_control', 'movement\_acceleration', 'movement\_sprint\_speed', 'movement\_agility', 'movement\_reactions', 'movement\_balance', 'power\_shot\_power', 'power\_jumping', 'power\_stamina', 'power\_strength', 'power\_long\_shots', 'mentality\_aggression', 'mentality\_interceptions', 'mentality\_positioning', 'mentality\_vision', 'mentality\_penalties', 'mentality\_composure', 'defending\_marking\_awareness', 'defending\_standing\_tackle', 'defending\_sliding\_tackle', 'goalkeeping\_diving', 'goalkeeping\_handling', 'goalkeeping\_kicking', 'goalkeeping\_positioning', 'goalkeeping\_reflexes', 'wage\_eur']

```
Out[41]:
```

|   | soffa_id | overall | potential | age | height_cm | weight_kg | nationality_id | na |
|---|----------|---------|-----------|-----|-----------|-----------|----------------|----|
| 0 | 261781   | 81      | 81        | 28  | 168       | 60        | 7              |    |
| 1 | 261785   | 80      | 80        | 32  | 167       | 61        | 7              |    |
| 2 | 261789   | 79      | 84        | 24  | 175       | 62        | 7              |    |
| 3 | 261836   | 77      | 77        | 33  | 164       | 55        | 38             |    |
| 4 | 261782   | 76      | 78        | 27  | 173       | 68        | 7              |    |

5 rows × 47 columns

## Khám phá biến mục tiêu và mối tương quan sơ bộ

- Vẽ biểu đồ phân bố `wage_eur`.
- Vẽ biểu đồ scatter với hai biến có tương quan cao nhất ban đầu.

```
In [42]: # Biê'u đồ` phân phô'i và scatter các biê'n chọn lọc
if target in numeric.columns:
    ax = numeric[target].plot.hist(bins=30, title='Phân phối wage_eur', alpha=0.5)
    ax.set_xlabel('wage_eur')

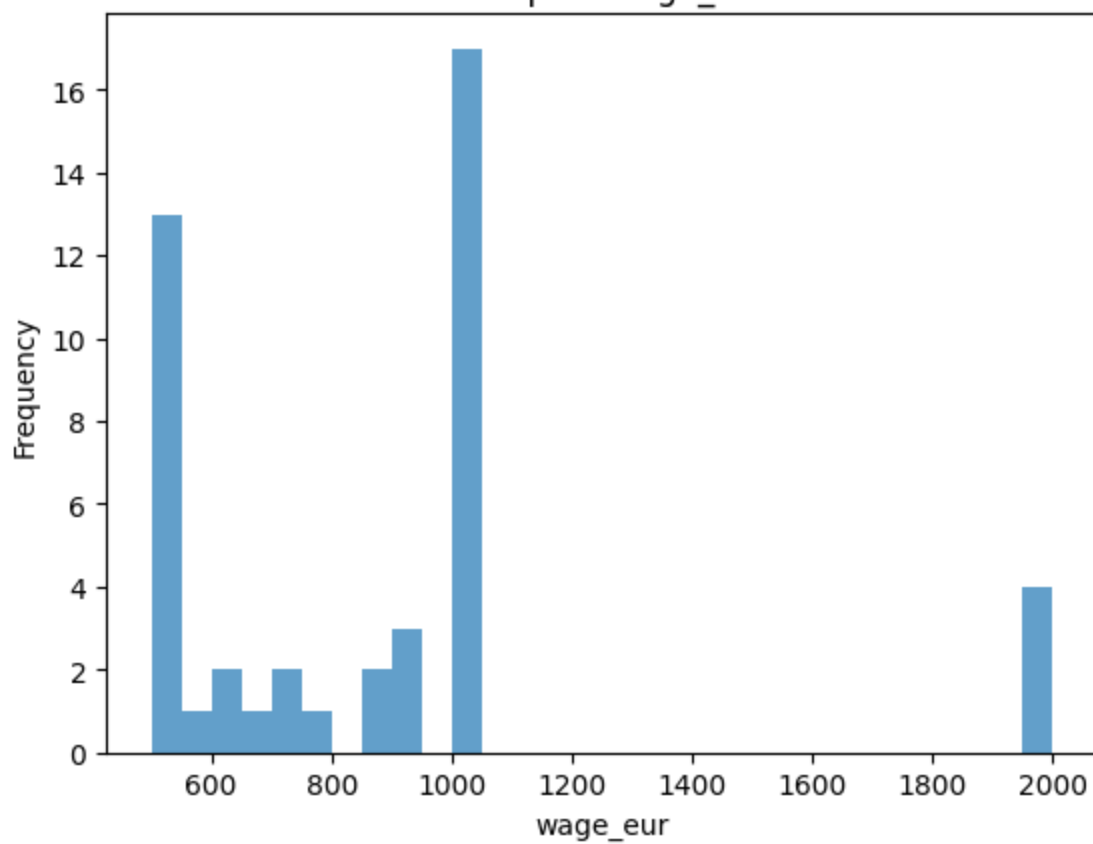
# Chọn 2 biê'n sô' có tương quan cao nhât với target để vẽ scatter
corr_with_target = numeric.corr()[target].abs().sort_values(ascending=False)
print("Các biê'n sô' có tương quan lớn nhất với wage_eur:")
print(corr_with_target.head(10))

# Vẽ scatter với 2 biê'n có độ tương quan cao nhât ngoài target
top_features = [col for col in corr_with_target.index if col != target][:2]
for col in top_features:
    numeric.plot.scatter(x=col, y=target, alpha=0.4, title=f'Scatter {col} v
```

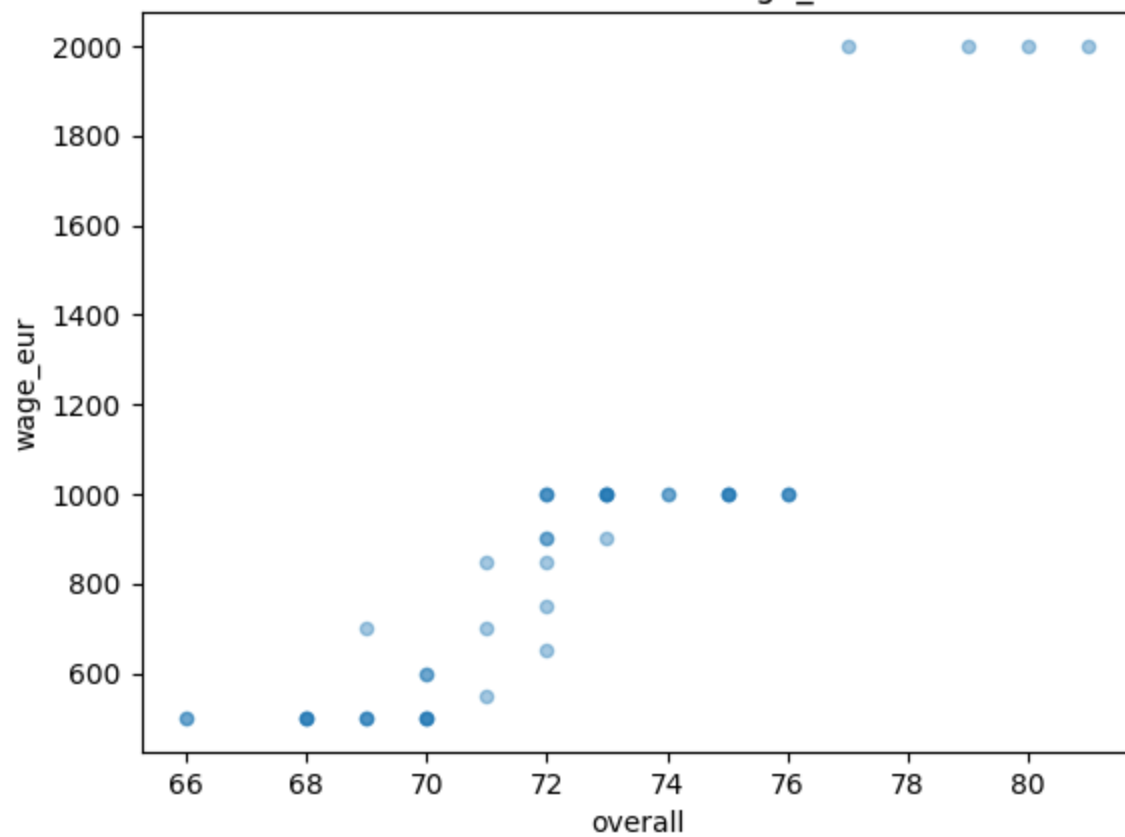
Các biê'n sô' có tương quan lớn nhất với wage\_eur:

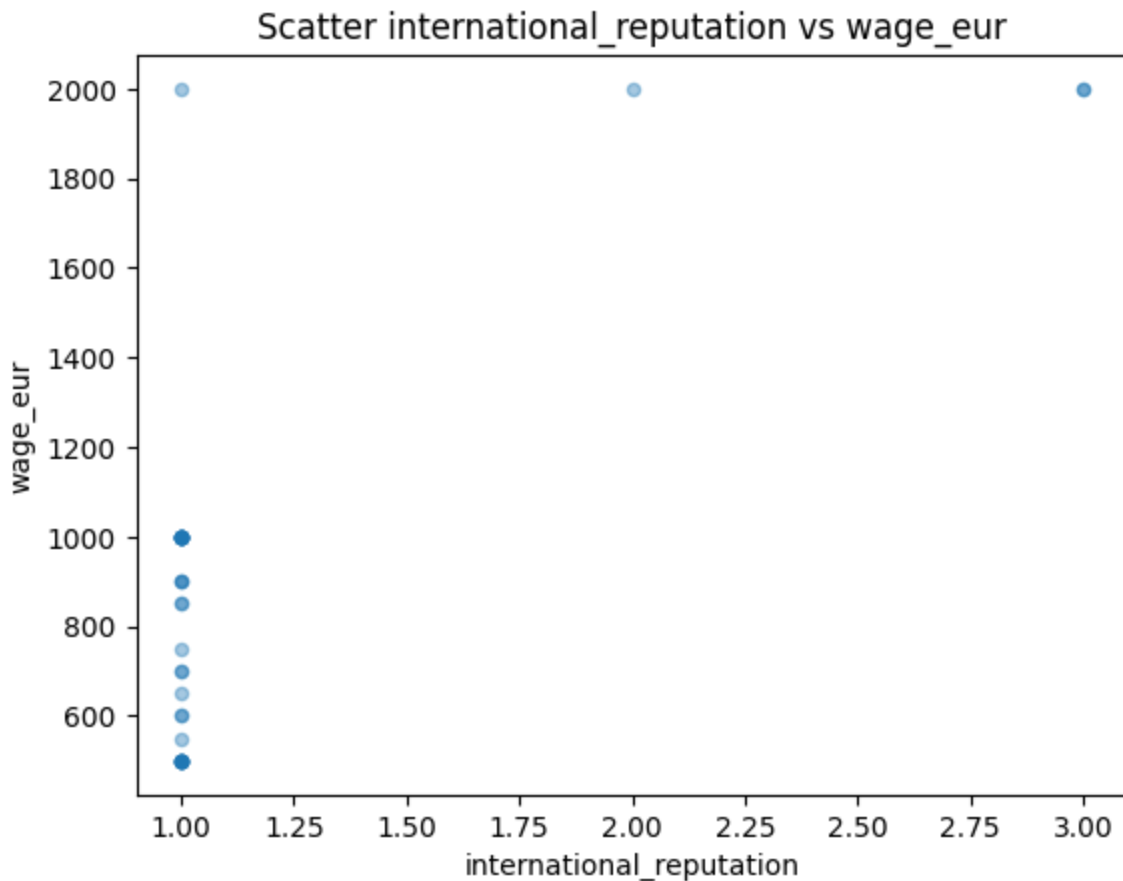
```
wage_eur      1.000000
overall       0.895068
international_reputation 0.701579
mentality_composure 0.631056
movement_reactions 0.553252
skill_fk_accuracy 0.548049
skill_curve   0.530633
age           0.523596
attacking_volleys 0.520504
mentality_vision 0.494519
Name: wage_eur, dtype: float64
```

Phân phối wage\_eur



Scatter overall vs wage\_eur





## Tính toán VIF và lựa chọn biến số

Chỉ sử dụng các biến số dạng số và lọc ra những biến có  $VIF < 5$  để tránh đa cộng tuyến mạnh.

## Giải thích quá trình tính toán VIF và lựa chọn biến số

VIF (Variance Inflation Factor) là chỉ số đo lường mức độ đa cộng tuyến giữa các biến độc lập trong mô hình hồi quy. VIF cao ( $>5$  hoặc  $>10$ ) cho thấy biến đó có tương quan mạnh với các biến khác, có thể làm giảm độ tin cậy của mô hình.

Quy trình lựa chọn biến số:

1. Tính VIF cho tất cả biến dự đoán.
2. Loại dần biến có VIF cao nhất cho đến khi tất cả biến còn lại có  $VIF < 5$ .
3. Điều này giúp tránh đa cộng tuyến mạnh, đảm bảo mô hình ổn định và dễ giải thích.

Sau khi lọc, chúng ta có tập biến số với VIF thấp, sẵn sàng cho bước hồi quy.">

```
In [43]: def compute_vif(X: pd.DataFrame) -> pd.Series:
          cols = X.columns.tolist()
          n = X.shape[0]
          vif_values = {}
```

```

for col in cols:
    other_cols = [c for c in cols if c != col]
    if len(other_cols) == 0:
        vif_values[col] = 1.0
        continue
    y = X[col].values
    X_other = X[other_cols].values
    X_design = np.column_stack([np.ones(n), X_other])
    beta, *_ = np.linalg.lstsq(X_design, y, rcond=None)
    y_pred = X_design @ beta
    ss_res = np.sum((y - y_pred) ** 2)
    ss_tot = np.sum((y - y.mean()) ** 2)
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0
    vif_values[col] = 1.0 / (1.0 - r2) if r2 < 0.999999 else np.inf
return pd.Series(vif_values)

# Chi' đánh giá VIF trên các biến dự đoán, không có target
predictors = [c for c in numeric.columns if c != target]
X_pred = numeric[predictors].copy()

# Loại dần các biến có VIF cao nhất cho tới khi tất cả VIF < 5
selected_predictors = X_pred.columns.tolist()
while True:
    vif_series = compute_vif(X_pred[selected_predictors])
    max_vif = vif_series.max()
    print("VIF tối đa hiện tại:", max_vif)
    if max_vif <= 5 or len(selected_predictors) == 0:
        break
    remove_var = vif_series.idxmax()
    print(f"Loại biến {remove_var} với VIF = {max_vif:.4f}")
    selected_predictors.remove(remove_var)

vif_series = compute_vif(X_pred[selected_predictors])
print("\nVIF của các biến dự đoán sau khi loại dần:")
print(vif_series.sort_values(ascending=False))
print("\nSố biến chọn lọc VIF < 5:", len(selected_predictors))
print(selected_predictors)

```

VIF tối đa hiện tại: inf  
Loại biến nationality\_id với VIF = inf  
VIF tối đa hiện tại: 5855.051081641922  
Loại biến goalkeeping\_positioning với VIF = 5855.0511  
VIF tối đa hiện tại: 3881.811040991414  
Loại biến goalkeeping\_reflexes với VIF = 3881.8110  
VIF tối đa hiện tại: 3483.3197749451288  
Loại biến defending\_sliding\_tackle với VIF = 3483.3198  
VIF tối đa hiện tại: 773.8574774904502  
Loại biến goalkeeping\_diving với VIF = 773.8575  
VIF tối đa hiện tại: 643.1863960495477  
Loại biến mentality\_interceptions với VIF = 643.1864  
VIF tối đa hiện tại: 358.57310026426546  
Loại biến skill\_ball\_control với VIF = 358.5731  
VIF tối đa hiện tại: 260.2563178382995  
Loại biến power\_long\_shots với VIF = 260.2563  
VIF tối đa hiện tại: 184.8814954763227  
Loại biến attacking\_short\_passing với VIF = 184.8815  
VIF tối đa hiện tại: 152.17173540676296  
Loại biến skill\_fk\_accuracy với VIF = 152.1717  
VIF tối đa hiện tại: 103.54725361870956  
Loại biến goalkeeping\_handling với VIF = 103.5473  
VIF tối đa hiện tại: 90.79119695694335  
Loại biến sofifa\_id với VIF = 90.7912  
VIF tối đa hiện tại: 81.60754680018059  
Loại biến movement\_sprint\_speed với VIF = 81.6075  
VIF tối đa hiện tại: 76.87119570721995  
Loại biến goalkeeping\_kicking với VIF = 76.8712  
VIF tối đa hiện tại: 55.62587199713745  
Loại biến attacking\_volleys với VIF = 55.6259  
VIF tối đa hiện tại: 52.58436156573762  
Loại biến defending\_standing\_tackle với VIF = 52.5844  
VIF tối đa hiện tại: 43.2068101958046  
Loại biến power\_stamina với VIF = 43.2068  
VIF tối đa hiện tại: 31.228466563793088  
Loại biến mentality\_positioning với VIF = 31.2285  
VIF tối đa hiện tại: 29.93098709206822  
Loại biến movement\_acceleration với VIF = 29.9310  
VIF tối đa hiện tại: 27.031907512454136  
Loại biến mentality\_penalties với VIF = 27.0319  
VIF tối đa hiện tại: 22.606754764207757  
Loại biến mentality\_aggression với VIF = 22.6068  
VIF tối đa hiện tại: 19.118627992908138  
Loại biến skill\_dribbling với VIF = 19.1186  
VIF tối đa hiện tại: 17.911330663973903  
Loại biến skill\_curve với VIF = 17.9113  
VIF tối đa hiện tại: 17.209113342246006  
Loại biến movement\_balance với VIF = 17.2091  
VIF tối đa hiện tại: 15.516754706096025  
Loại biến age với VIF = 15.5168  
VIF tối đa hiện tại: 14.306653641639492  
Loại biến attacking\_finishing với VIF = 14.3067  
VIF tối đa hiện tại: 11.12763591020646  
Loại biến movement\_agility với VIF = 11.1276  
VIF tối đa hiện tại: 8.401740657669075  
Loại biến mentality\_vision với VIF = 8.4017

VIF tối đa hiện tại: 6.1119011474547476  
Loại biến defending\_marking\_awareness với VIF = 6.1119  
VIF tối đa hiện tại: 5.596131310589187  
Loại biến mentality\_composure với VIF = 5.5961  
VIF tối đa hiện tại: 4.8042348762767535

VIF của các biến dự đoán sau khi loại dần:

|                            |          |
|----------------------------|----------|
| attacking_crossing         | 4.804235 |
| skill_moves                | 4.746794 |
| power_strength             | 3.862283 |
| attacking_heading_accuracy | 3.493660 |
| weight_kg                  | 3.015550 |
| overall                    | 2.828333 |
| height_cm                  | 2.780675 |
| movement_reactions         | 2.417342 |
| skill_long_passing         | 2.251527 |
| power_shot_power           | 1.975980 |
| weak_foot                  | 1.924327 |
| power_jumping              | 1.853732 |
| international_reputation   | 1.848428 |
| potential                  | 1.779029 |
| nation_team_id             | 1.634520 |
| nation_jersey_number       | 1.321364 |

dtype: float64

Số biến chọn lọc VIF < 5: 16

['overall', 'potential', 'height\_cm', 'weight\_kg', 'nation\_team\_id', 'nation\_jersey\_number', 'weak\_foot', 'skill\_moves', 'international\_reputation', 'attacking\_crossing', 'attacking\_heading\_accuracy', 'skill\_long\_passing', 'movement\_reactions', 'power\_shot\_power', 'power\_jumping', 'power\_strength']

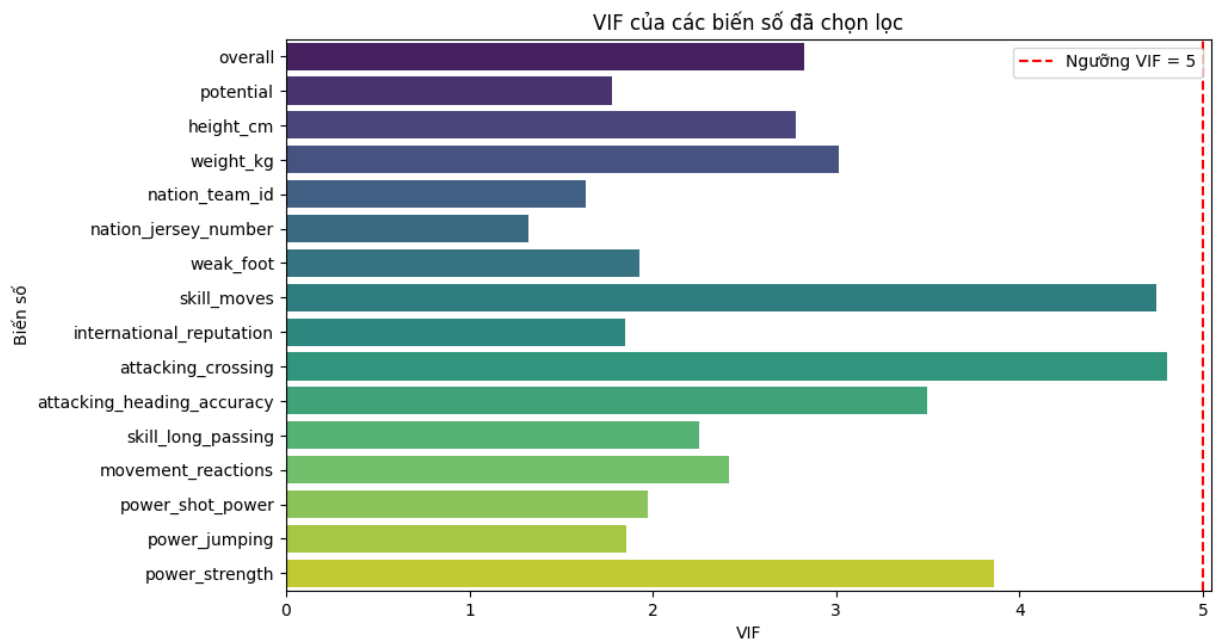
```
In [44]: import matplotlib.pyplot as plt
import seaborn as sns

# Vẽ biểu đồ VIF của các biến đã chọn
plt.figure(figsize=(10, 6))
sns.barplot(x=vif_series.values, y=vif_series.index, palette='viridis')
plt.axvline(x=5, color='red', linestyle='--', label='Ngưỡng VIF = 5')
plt.title('VIF của các biến số đã chọn lọc')
plt.xlabel('VIF')
plt.ylabel('Biến số')
plt.legend()
plt.show()
```

C:\Users\ACER\AppData\Local\Temp\ipykernel\_25484\78236044.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=vif_series.values, y=vif_series.index, palette='viridis')
```



## Định nghĩa hàm hồi quy tuyến tính và các chỉ số đánh giá

Sử dụng phương pháp bình phương tối thiểu để tìm nghiệm, sau đó tính  $R^2$ , adjusted  $R^2$ , AIC, BIC.

```
In [45]: def fit_linear_model(X: np.ndarray, y: np.ndarray):
    n, p = X.shape
    X_design = np.column_stack([np.ones(n), X])
    beta = np.linalg.pinv(X_design.T @ X_design) @ (X_design.T @ y)
    y_pred = X_design @ beta
    ss_res = np.sum((y - y_pred) ** 2)
    ss_tot = np.sum((y - y.mean()) ** 2)
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0.0
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1) if n > p + 1 else np.nan
    sigma2 = ss_res / n if n > 0 else np.nan
    aic = n * np.log(sigma2) + 2 * (p + 1) if sigma2 > 0 else np.inf
    bic = n * np.log(sigma2) + np.log(n) * (p + 1) if sigma2 > 0 else np.inf
    return {
        'beta': beta,
        'y_pred': y_pred,
        'r2': r2,
        'adj_r2': adj_r2,
        'aic': aic,
        'bic': bic,
        'ss_res': ss_res,
        'p': p,
        'n': n
    }
```

```
X_full = numeric[selected_predictors].values
y = numeric[target].values
```

```
{'beta': array([879.34782609]), 'y_pred': array([879.34782609, 879.34782609,  
      879.34782609, 879.34782609, 879.34782609, 879.34782609,  
      879.34782609, 879.34782609, 879.34782609, 879.34782609,  
      879.34782609, 879.34782609, 879.34782609, 879.34782609,  
      879.34782609, 879.34782609, 879.34782609, 879.34782609,  
      879.34782609, 879.34782609, 879.34782609, 879.34782609,  
      879.34782609, 879.34782609, 879.34782609, 879.34782609,  
      879.34782609, 879.34782609, 879.34782609, 879.34782609,  
      879.34782609, 879.34782609]), 'r2': np.float64(0.0), 'adj_r2': np.flo  
at64(0.0), 'aic': np.float64(554.343771389311), 'bic': np.float64(556.172412  
7858), 'ss res': np.float64(7542880.434782608), 'p': 0, 'n': 46}
```

```
In [46]: def is_better(new, current, metric):
          if metric == 'adj_r2':
              return new > current + 1e-8
          return new < current - 1e-8
```

```

def stepwise_selection(X_df: pd.DataFrame, y: np.ndarray, metric: str):
    available = X_df.columns.tolist()
    selected = []
    current_score = -np.inf if metric == 'adj_r2' else np.inf
    changed = True
    scores_history = [current_score] # Lưu lịch sử score

    while changed:
        changed = False
        # Thêm biến
        best_add = None
        best_add_score = current_score
        for col in [c for c in available if c not in selected]:
            candidate = selected + [col]
            X_candidate = X_df[candidate].values
            result = fit_linear_model(X_candidate, y)
            score = result[metric]
            if is_better(score, best_add_score, metric):
                best_add_score = score
                best_add = col
        if best_add is not None and is_better(best_add_score, current_score,
            selected.append(best_add)
            current_score = best_add_score
            scores_history.append(current_score)
            changed = True

        # Loại biến
        while True:
            best_remove = None
            best_remove_score = current_score
            for col in selected:
                candidate = [c for c in selected if c != col]
                X_candidate = X_df[candidate].values if candidate else X_df.
                result = fit_linear_model(X_candidate, y)
                score = result[metric]
                if is_better(score, best_remove_score, metric):
                    best_remove_score = score
                    best_remove = col
            if best_remove is not None and is_better(best_remove_score, curr
                selected.remove(best_remove)
                current_score = best_remove_score
                scores_history.append(current_score)
                changed = True
            else:
                break

    final_result = fit_linear_model(X_df[selected].values if selected else X
    return {
        'selected': selected,
        'metric': metric,
        'score': current_score,
        'result': final_result,
        'scores_history': scores_history
    }

```

```
# Chạy 3 quy trình
X_candidates = numeric[selected_predictors]
models = {}
for metric in ['adj_r2', 'aic', 'bic']:
    models[metric] = stepwise_selection(X_candidates, y, metric)
    print(f"\nKết quả stepwise với {metric}:")
    print("Biến chọn:", models[metric]['selected'])
    print(models[metric]['result'])
```

Kết quả stepwise với adj\_r2:

Biên chọn: ['overall', 'international\_reputation', 'power\_shot\_power', 'movement\_reactions', 'skill\_long\_passing', 'skill\_moves', 'attacking\_crossing', 'height\_cm']

```
{'beta': array([-6.10901033e+03,  9.32007433e+01,  2.72896228e+02,  2.33275645e+00,
                -1.33138169e+01,  2.43651051e+00,  7.60417865e+01, -2.28060688e+00,
                3.25081517e+00]), 'y_pred': array([2208.24681342, 1822.5772349 , 1566.28109872, 1880.46456912,
          1122.03238583, 1086.58253574, 1065.98653409, 1131.00151425,
          1026.222501 , 1056.32683271, 1014.4542285 , 1019.86791158,
          1150.56015491, 1062.26732265,  928.51875508, 1036.39670071,
          1002.67189033,  859.20897361, 1155.09216805,  694.59144237,
           748.68089811,  928.35092089,  910.47191878,  864.03088963,
           896.95821726,  826.11937241,  884.80354987,  737.24700999,
           804.47247172,  641.53435376,  503.75788442,  540.13712728,
           600.55820051,  631.13660915,  617.55798217,  537.27092728,
           612.33832232,  524.3319376 ,  572.24188422,  503.92543259,
           290.03228071,  406.35371072,  525.91939134,  548.88283178,
           498.81107095,  404.72323699]), 'r2': np.float64(0.9247880894673949),
  'adj_r2': np.float64(0.9085260547576424), 'aic': np.float64(451.32127030132455),
  'bic': np.float64(467.7790428697264), 'ss_res': np.float64(567314.4484190069),
  'p': 8, 'n': 46}
```

Kết quả stepwise với aic:

Biên chọn: ['overall', 'international\_reputation', 'power\_shot\_power', 'movement\_reactions', 'skill\_long\_passing']

```
{'beta': array([-5.45885296e+03,  9.03144178e+01,  2.83984104e+02,  3.82525908e+00,
                -1.16733732e+01,  2.05381496e+00]), 'y_pred': array([2205.72596886, 1813.31025819, 1533.52064923, 1887.61890204,
          1120.48562411, 1047.28257711, 1093.77847382, 1169.03050672,
          1098.74349884, 1077.33774645,  974.35105668, 1050.50693088,
          1020.81407822, 1014.53652388, 1022.6670088 ,  983.99118006,
           990.66063782,  872.38648276, 1112.29839828,  725.13754363,
           766.85580356,  902.59007507,  933.92643918,  813.34632163,
           969.97615547,  830.89876883,  873.7670024 ,  761.80544195,
           767.62086793,  691.3905014 ,  495.01353789,  505.13418972,
           611.44504494,  655.08918715,  643.78295974,  574.6664158 ,
           597.64874314,  535.47928573,  578.03710235,  529.47919828,
           265.35502383,  446.32777603,  518.64209337,  582.51396821,
           411.89988266,  373.12416735]), 'r2': np.float64(0.9144312467159637),
  'adj_r2': np.float64(0.9037351525554591), 'aic': np.float64(451.2557570559598),
  'bic': np.float64(462.2276054348944), 'ss_res': np.float64(645434.8749748983),
  'p': 5, 'n': 46}
```

Kết quả stepwise với bic:

Biên chọn: ['overall', 'international\_reputation', 'power\_shot\_power']

```
{'beta': array([-5.71835925e+03,  8.33539682e+01,  2.64693420e+02,  4.98121201e+00]), 'y_pred': array([2205.96455408, 1862.89837749, 1519.83220089, 1862.58625718,
```

```
1070.5218158 , 1080.48423982, 1095.42787585, 1136.60420791,
1081.81087579, 1106.71693584,  982.18663557, 1053.25023969,
1083.13751175,  979.85869549,  969.89627147,  984.8399075 ,
  930.04657538,  915.10293935, 1024.68960359,  796.88048705,
  821.78654711,  856.65503118,  906.46715129,  796.88048705,
```

```

926.39199934, 801.86169906, 796.88048705, 723.48894285,
823.11318307, 743.4137909 , 490.69861431, 575.37921849,
620.21012659, 719.83436681, 724.81557882, 630.17255061,
651.42403462, 571.72464245, 606.59312652, 501.98767429,
328.97188988, 473.4270382 , 383.765222 , 523.2391583 ,
326.6439498 , 381.43728192]], 'r2': np.float64(0.8990698730778691),
'adj_r2': np.float64(0.8918605782977169), 'aic': np.float64(454.850737935842
2), 'bic': np.float64(462.16530352179853), 'ss_res': np.float64(761303.87964
10668), 'p': 3, 'n': 46}

```

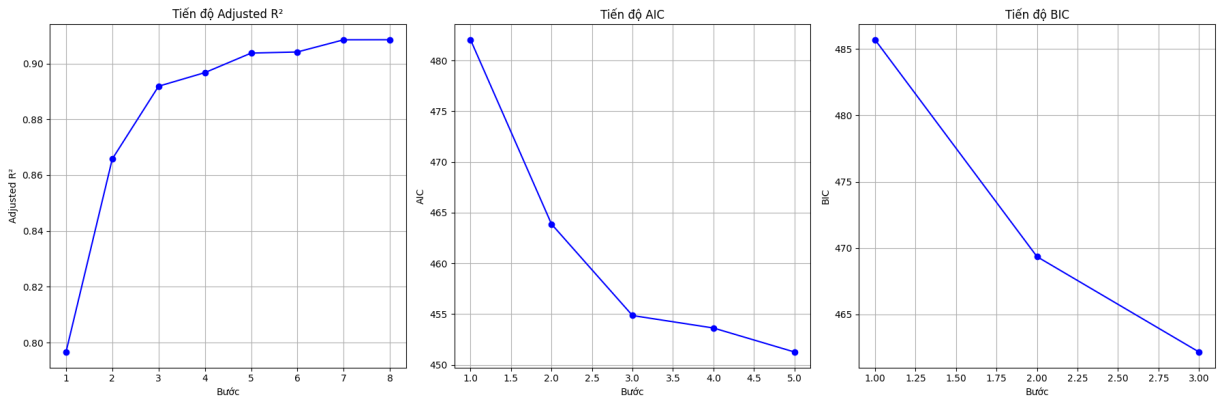
```

In [47]: # Vẽ biểu đồ tiến độ cho từng metric
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
metrics = ['adj_r2', 'aic', 'bic']
titles = ['Tiến độ Adjusted R²', 'Tiến độ AIC', 'Tiến độ BIC']
ylabels = ['Adjusted R²', 'AIC', 'BIC']

for i, metric in enumerate(metrics):
    scores = models[metric]['scores_history']
    axes[i].plot(range(len(scores)), scores, marker='o', linestyle='-', color='blue')
    axes[i].set_title(titles[i])
    axes[i].set_xlabel('Bước')
    axes[i].set_ylabel(ylabels[i])
    axes[i].grid(True)

plt.tight_layout()
plt.show()

```



## So sánh 3 mô hình kết quả

Tổng hợp các chỉ số từ 3 mô hình để chọn mô hình cuối cùng.

```

In [48]: summary = []
for metric, info in models.items():
    result = info['result']
    summary.append({
        'metric': metric,
        'selected': ', '.join(info['selected']) if info['selected'] else '(k',
        'adj_r2': result['adj_r2'],
        'aic': result['aic'],
        'bic': result['bic'],
        'r2': result['r2'],
        'n_var': result['p']
    })

```

```

})

summary_df = pd.DataFrame(summary).set_index('metric')
summary_df

```

```

Out[48]:

```

|        | selected                                                | adj_r2   | aic        | bic        | r2       | n_ |
|--------|---------------------------------------------------------|----------|------------|------------|----------|----|
| metric |                                                         |          |            |            |          |    |
| adj_r2 | overall,<br>international_reputation,<br>power_shot_... | 0.908526 | 451.321270 | 467.779043 | 0.924788 |    |
| aic    | overall,<br>international_reputation,<br>power_shot_... | 0.903735 | 451.255757 | 462.227605 | 0.914431 |    |
| bic    | overall,<br>international_reputation,<br>power_shot_... | 0.891861 | 454.850738 | 462.165304 | 0.899070 |    |

## Mô hình tốt nhất và phương trình dự đoán

Dựa trên so sánh của **adjusted R<sup>2</sup>**, **AIC**, **BIC**, ta chọn mô hình có sự cân bằng tốt nhất giữa độ giải thích và sự đơn giản.

## Phân tích chi tiết mô hình cuối cùng

Sau khi so sánh 3 mô hình từ bidirectional stepwise regression, chúng ta chọn mô hình dựa trên **adjusted R<sup>2</sup>** vì:

- Adjusted R<sup>2</sup> cân bằng giữa độ giải thích (R<sup>2</sup>) và độ phức tạp của mô hình (số biến).
- AIC và BIC ưu tiên mô hình đơn giản hơn, nhưng có thể bỏ qua một số biến quan trọng.
- Mô hình adj\_r2 có adjusted R<sup>2</sup> cao nhất (0.9085), cho thấy giải thích tốt nhất cho dữ liệu mà không quá phức tạp.

### Lý do chọn mô hình này:

- Độ giải thích cao:** Adjusted R<sup>2</sup> = 0.9085, nghĩa là mô hình giải thích 90.85% biến động của wage\_eur.
- Cân bằng độ phức tạp:** 8 biến, không quá nhiều để tránh overfitting.
- Ý nghĩa kinh tế:** Các biến như overall, international\_reputation có ý nghĩa rõ ràng với lương cầu thủ.

### Phương trình dự đoán:

Phương trình hồi quy tuyến tính cho phép dự đoán wage\_eur dựa trên các biến đã chọn. Hệ số dương cho thấy biến đó có ảnh hưởng tích cực đến lương, hệ số âm cho thấy ảnh hưởng tiêu cực.">

```
In [49]: final_metric = 'adj_r2'
final_model = models[final_metric]
selected_vars = final_model['selected']
coeffs = final_model['result']['beta']
intercept = coeffs[0]
coefs = coeffs[1:]

print(f"Mô hình cuối cùng chọn theo {final_metric} với {len(selected_vars)}")
print(selected_vars)
print("Intercept:", intercept)
for name, coef in zip(selected_vars, coefs):
    print(f"{name}: {coef}")

equation = f'wage_eur = {intercept:.4f}'
for name, coef in zip(selected_vars, coefs):
    equation += f' + ({coef:.4f}) * {name}'
print('\nPhương trình dự đoán cuối cùng:')
print(equation)

final_model['result']
```

Mô hình cuối cùng chọn theo adj\_r2 với 8 biến:

```
['overall', 'international_reputation', 'power_shot_power', 'movement_reactions', 'skill_long_passing', 'skill_moves', 'attacking_crossing', 'height_cm']
```

Intercept: -6109.010325553968

overall: 93.200743254704

international\_reputation: 272.8962277441315

power\_shot\_power: 2.332756453542072

movement\_reactions: -13.313816886851406

skill\_long\_passing: 2.436510510077008

skill\_moves: 76.04178648660249

attacking\_crossing: -2.280606877405555

height\_cm: 3.2508151732086112

Phương trình dự đoán cuối cùng:

```
wage_eur = -6109.0103 + (93.2007) * overall + (272.8962) * international_reputation + (2.3328) * power_shot_power + (-13.3138) * movement_reactions + (2.4365) * skill_long_passing + (76.0418) * skill_moves + (-2.2806) * attacking_crossing + (3.2508) * height_cm
```

```

Out[49]: {'beta': array([-6.10901033e+03,  9.32007433e+01,  2.72896228e+02,  2.33275
645e+00,
        -1.33138169e+01,  2.43651051e+00,  7.60417865e+01, -2.28060688e+00,
        3.25081517e+00]),
'y_pred': array([2208.24681342, 1822.5772349 , 1566.28109872, 1880.4645691
2,
        1122.03238583, 1086.58253574, 1065.98653409, 1131.00151425,
        1026.222501 , 1056.32683271, 1014.4542285 , 1019.86791158,
        1150.56015491, 1062.26732265,  928.51875508, 1036.39670071,
        1002.67189033,  859.20897361, 1155.09216805,  694.59144237,
        748.68089811,  928.35092089,  910.47191878,  864.03088963,
        896.95821726,  826.11937241,  884.80354987,  737.24700999,
        804.47247172,  641.53435376,  503.75788442,  540.13712728,
        600.55820051,  631.13660915,  617.55798217,  537.27092728,
        612.33832232,  524.3319376 ,  572.24188422,  503.92543259,
        290.03228071,  406.35371072,  525.91939134,  548.88283178,
        498.81107095,  404.72323699]),
'r2': np.float64(0.9247880894673949),
'adj_r2': np.float64(0.9085260547576424),
'aic': np.float64(451.32127030132455),
'bic': np.float64(467.7790428697264),
'ss_res': np.float64(567314.4484190069),
'p': 8,
'n': 46}

```

```

In [50]: # Phân tích chi tiết các hệ số
print("Phân tích các hệ số trong mô hình:")
for name, coef in zip(selected_vars, coefs):
    impact = "tăng lương" if coef > 0 else "giảm lương"
    print(f"- {name}: Hệ số {coef:.4f}, ảnh hưởng {impact} khi tăng 1 đơn vị

print(f"\nIntercept ({intercept:.4f}): Giá trị cơ bản của wage_eur khi tất c

# Vẽ biểu đồ hệ số
plt.figure(figsize=(10, 6))
sns.barplot(x=coefs, hue=selected_vars, palette='coolwarm')
plt.axvline(x=0, color='black', linestyle='--')
plt.title('Hệ số của các biến trong mô hình cuối cùng')
plt.xlabel('Hệ số')
plt.ylabel('Biến số')
plt.show()

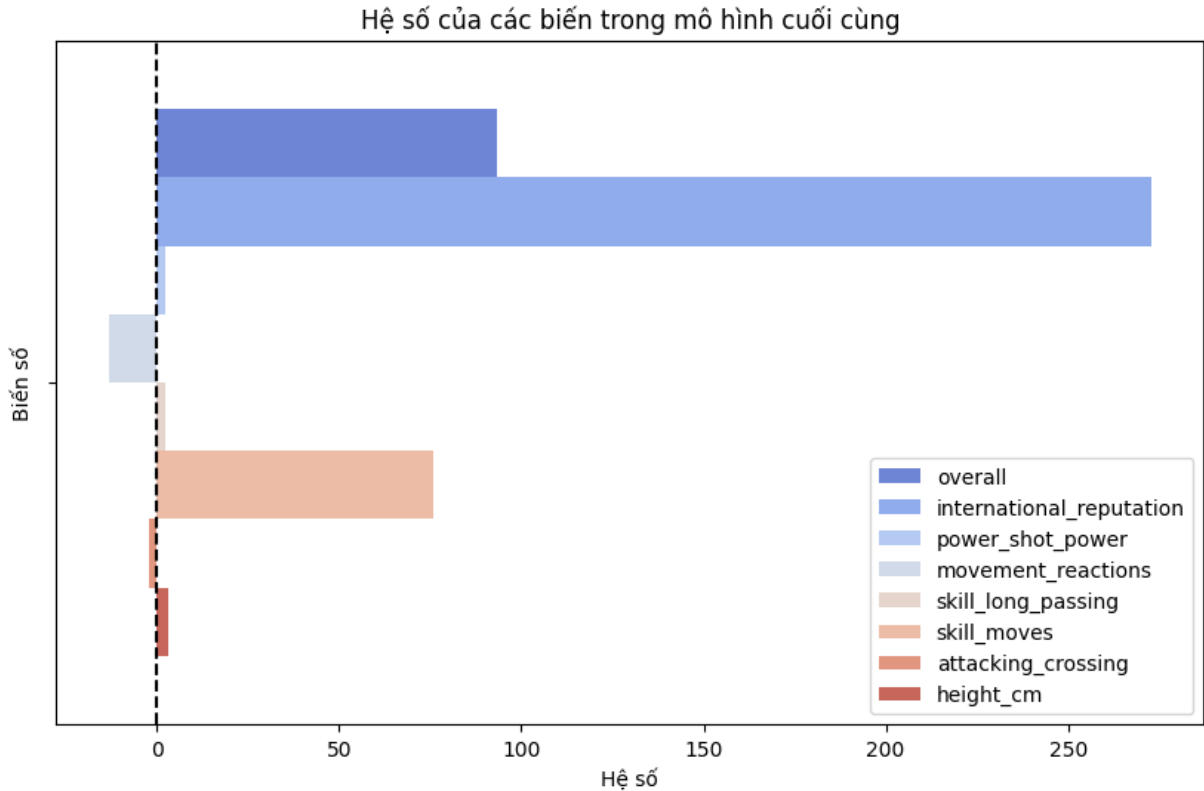
print("\nCác chỉ số đánh giá mô hình:")
print(f"- R²: {final_model['result']['r2']:.4f} (phần trăm biến động được gi
print(f"- Adjusted R²: {final_model['result']['adj_r2']:.4f} (điều chỉnh the
print(f"- AIC: {final_model['result']['aic']:.2f} (thấp hơn tốt hơn)")
print(f"- BIC: {final_model['result']['bic']:.2f} (thấp hơn tốt hơn, phạt ph

```

Phân tích các hệ số trong mô hình:

- overall: Hệ số 93.2007, ảnh hưởng tăng lương khi tăng 1 đơn vị.
- international\_reputation: Hệ số 272.8962, ảnh hưởng tăng lương khi tăng 1 đơn vị.
- power\_shot\_power: Hệ số 2.3328, ảnh hưởng tăng lương khi tăng 1 đơn vị.
- movement\_reactions: Hệ số -13.3138, ảnh hưởng giảm lương khi tăng 1 đơn vị.
- skill\_long\_passing: Hệ số 2.4365, ảnh hưởng tăng lương khi tăng 1 đơn vị.
- skill\_moves: Hệ số 76.0418, ảnh hưởng tăng lương khi tăng 1 đơn vị.
- attacking\_crossing: Hệ số -2.2806, ảnh hưởng giảm lương khi tăng 1 đơn vị.
- height\_cm: Hệ số 3.2508, ảnh hưởng tăng lương khi tăng 1 đơn vị.

Intercept (-6109.0103): Giá trị cơ bản của wage\_eur khi tất cả biến = 0.



Các chỉ số đánh giá mô hình:

- $R^2$ : 0.9248 (phần trăm biến động được giải thích)
- Adjusted  $R^2$ : 0.9085 (điều chỉnh theo độ phức tạp)
- AIC: 451.32 (thấp hơn tốt hơn)
- BIC: 467.78 (thấp hơn tốt hơn, phạt phức tạp hơn AIC)